

3. Магические методы (ака Перегрузки)

Введение: перегрузка операторов в программировании

Что такое перегрузка и зачем она нужна?

В предыдущих лекциях мы научились создавать классы и объекты, определять их состояние и поведение. Но представьте, что у нас есть два объекта класса `Vector` (математический вектор), и мы хотим сложить их друг с другом. Если мы попробуем написать `v1 + v2`, Python (как и большинство языков) выдаст ошибку, потому что он не знает, как складывать наши кастомные типы.

Перегрузка операторов — это механизм, позволяющий определить поведение встроенных операторов языка (например, `+`, `-`, `*`, `==`, `[]`, `()`) для пользовательских типов данных. Это делает код более интуитивным и читаемым, приближая работу с объектами к работе со встроенными типами.

Аналогия: представьте, что вы учите робота готовить. Вы говорите ему: «Смешай муку и воду». Робот понимает, что значит «смешать» для жидкостей, но если вы попросите «смешай гитару и стул», он будет в замешательстве. Перегрузка — это когда мы объясняем роботу: «Когда видишь гитару и стул, смешивание означает положить гитару на стул».

Перегрузка в классических статически типизированных языках

Чтобы понять особенности реализации в Python, давайте посмотрим, как перегрузка выглядит в языках, где она появилась исторически.

C++

В C++ перегрузка операторов — это синтаксическая возможность, позволяющая определить специальные функции-члены класса с ключевым словом `operator`.

Например, перегрузка сложения для класса `Vector`:

```
class Vector {
public:
    double x, y;

    Vector(double x, double y) : x(x), y(y) {}

    // Перегрузка оператора +
    Vector operator+(const Vector& other) const {
        return Vector(x + other.x, y + other.y);
    }
};

Vector v1(1, 2), v2(3, 4);
```

```
Vector v3 = v1 + v2; // v3.x == 4, v3.y == 6
```

В C++ можно перегрузить практически любой оператор, включая `+`, `-`, `*`, `/`, `==`, `!=`, `[]`, `()`, `->`, операторы приведения типов, операторы выделения памяти и т.д. При этом сохраняется строгая типизация: компилятор проверяет соответствие типов на этапе компиляции.

C#

C# унаследовал идею перегрузки операторов от C++, но с некоторыми упрощениями и ограничениями. Перегрузка выполняется через статические методы с ключевым словом `operator`:

```
public class Vector {  
    public double X { get; set; }  
    public double Y { get; set; }  
  
    public Vector(double x, double y) { X = x; Y = y; }  
  
    public static Vector operator +(Vector a, Vector b) {  
        return new Vector(a.X + b.X, a.Y + b.Y);  
    }  
}
```

В отличие от C++, в C# перегружаемые операторы должны быть объявлены как `public static`, и некоторые операторы (например, `[]`) не могут быть перегружены напрямую (вместо них используются индексаторы).

Pascal (Object Pascal / Delphi)

В диалектах Object Pascal перегрузка операторов появилась позже и имеет свой синтаксис:

```
type  
    TVector = record  
        X, Y: Double;  
  
        class operator Add(a, b: TVector): TVector;  
    end;  
  
class operator TVector.Add(a, b: TVector): TVector;  
begin  
    Result.X := a.X + b.X;  
    Result.Y := a.Y + b.Y;
```

```
end;
```

Общие черты классической перегрузки

- **Статическая типизация:** компилятор точно знает, какой оператор вызывается на основе типов операндов.
- **Специальный синтаксис:** ключевое слово `operator` и строго определённые сигнатуры методов.
- **Ограничения на перегрузку:** не все операторы можно перегрузить, некоторые имеют обязательные парные перегрузки (например, если перегружен `==`, обычно требуется перегрузить и `!=`).
- **Приоритет операторов сохраняется** и не может быть изменён программистом.

Магические методы в Python

Как Python отличается от классических языков?

Python, будучи языком с динамической типизацией, пошёл другим путём. Вместо ключевого слова `operator` и статических сигнатур, он использует специальные методы с заранее определёнными именами, которые окружены двумя подчёркиваниями с обеих сторон: `__add__`, `__eq__`, `__str__` и т.д. Эти методы называют **магическими** (magic methods) или **дандер-методами** (от *double underscore* — dunder).

Когда интерпретатор Python встречает выражение `a + b`, он не проверяет типы на этапе компиляции, а динамически ищет метод `__add__` у объекта `a` и вызывает его, передавая `b` как аргумент. Если метод не найден, то ищется `__radd__` у `b` (об этом позже). Если ни один не определён, генерируется `TypeError`.

Таким образом, магические методы в Python — это **реализация перегрузки операторов через соглашение об именах**, а не через специальный синтаксис языка. Это делает перегрузку более гибкой и однородной с остальными методами класса.

Сходства и различия

Характеристика	Классическая перегрузка (C++/C#)	Магические методы Python
Способ определения	Ключевое слово <code>operator</code>	Методы с именем <code>__имя__</code>
Типизация	Статическая, проверка на этапе компиляции	Динамическая, поиск метода во время выполнения
Место определения	Методы-члены (или статические в C#)	Обычные методы экземпляра
Гибкость	Строгие сигнатуры	Можно реализовать любую логику, возвращать любой тип

Характеристика	Классическая перегрузка (C++/C#)	Магические методы Python
Ограничения	Не все операторы перегружаемы, некоторые требуют парности	Можно перегрузить почти всё, что имеет соответствующий магический метод
Наследование	Наследуются как обычные методы	Наследуются, поддерживается полиморфизм

Как работают магические методы в Python?

Когда вы пишете выражение с оператором, интерпретатор транслирует его в вызов соответствующего магического метода. Вот несколько примеров:

Выражение	Эквивалентный вызов
<code>x + y</code>	<code>x.__add__(y)</code>
<code>x - y</code>	<code>x.__sub__(y)</code>
<code>x * y</code>	<code>x.__mul__(y)</code>
<code>x == y</code>	<code>x.__eq__(y)</code>
<code>x < y</code>	<code>x.__lt__(y)</code>
<code>str(x)</code>	<code>x.__str__()</code>
<code>len(x)</code>	<code>x.__len__()</code>
<code>x[key]</code>	<code>x.__getitem__(key)</code>
<code>x(key)</code>	<code>x.__call__(key)</code>
<code>with x:</code>	<code>x.__enter__()</code> и <code>x.__exit__()</code>

Важно: Python не требует, чтобы эти методы возвращали значение строго определённого типа. Вы можете вернуть что угодно, но следует придерживаться здравого смысла, чтобы не нарушать ожидания пользователей вашего класса.

Основные группы магических методов

Давайте разберём наиболее часто используемые магические методы по группам.

1. Методы жизненного цикла объекта

С этими методами вы уже знакомы по прошлой лекции.

- `__init__(self, ...)` — конструктор, вызывается при создании экземпляра.
- `__new__(cls, ...)` — низкоуровневый метод, создающий сам объект перед `__init__`. Редко переопределяется.
- `__del__(self)` — деструктор, вызывается при удалении объекта сборщиком мусора. Использовать с осторожностью.

2. Методы представления объекта

Определяют, как объект будет выглядеть при преобразовании в строку.

- `__str__(self)` — вызывается функцией `str()`, `print()`, форматированием через f-строки. Должен возвращать «человеко-читаемое» представление.
- `__repr__(self)` — вызывается функцией `repr()`, в интерактивной оболочке. Должен возвращать строку, по возможности воссоздающую объект при передаче в `eval()`. Если `__str__` не определён, используется `__repr__`.

Пример:

```
class Point:

    def __init__(self, x, y):

        self.x = x

        self.y = y

    def __repr__(self):

        return f"Point({self.x}, {self.y})"

    def __str__(self):

        return f"({self.x}, {self.y})"

p = Point(3, 4)

print(p)          # (3, 4) — использует __str__

print(repr(p))    # Point(3, 4) — использует __repr__

p                # в Jupyter/IPython вызовет __repr__
```

3. Арифметические операторы

Для каждого бинарного арифметического оператора существует прямой магический метод и обратный (с префиксом `r`), который вызывается, если левый операнд не поддерживает операцию с правым.

Оператор	Прямой метод	Обратный метод
<code>+</code>	<code>__add__(self, other)</code>	<code>__radd__(self, other)</code>
<code>-</code>	<code>__sub__(self, other)</code>	<code>__rsub__(self, other)</code>
<code>*</code>	<code>__mul__(self, other)</code>	<code>__rmul__(self, other)</code>
<code>/</code>	<code>__truediv__(self, other)</code>	<code>__rtruediv__(self, other)</code>
<code>//</code>	<code>__floordiv__(self, other)</code>	<code>__rfloordiv__(self, other)</code>
<code>%</code>	<code>__mod__(self, other)</code>	<code>__rmod__(self, other)</code>
<code>**</code>	<code>__pow__(self, other)</code>	<code>__rpow__(self, other)</code>

Также есть методы для унарных операторов: `__neg__` (унарный минус), `__pos__` (унарный плюс), `__abs__` (абсолютное значение).

Пример для класса `Vector2D`:

```
class Vector2D:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        if isinstance(other, Vector2D):
            return Vector2D(self.x + other.x, self.y + other.y)
        return NotImplemented

    def __radd__(self, other):
        # Если складываем с неизвестным типом, пытаемся преобразовать
        if isinstance(other, (int, float)):
            return Vector2D(self.x + other, self.y + other)
        return NotImplemented
```

```
def __repr__(self):
```

```
    return f"Vector2D({self.x}, {self.y})"
```

```
v1 = Vector2D(1, 2)
```

```
v2 = Vector2D(3, 4)
```

```
print(v1 + v2)          # Vector2D(4, 6)
```

```
print(5 + v1)           # Vector2D(6, 7)
```

Замечание о `NotImplemented`: Если операция не поддерживается, метод должен вернуть специальный объект `NotImplemented` (не путать с исключением `NotImplementedError`). Это сигнал интерпретатору попробовать обратный метод у правого операнда.

4. Операторы сравнения

Позволяют объектам участвовать в операциях сравнения.

Оператор	Метод
<code>==</code>	<code>__eq__(self, other)</code>
<code>!=</code>	<code>__ne__(self, other)</code>
<code><</code>	<code>__lt__(self, other)</code>
<code><=</code>	<code>__le__(self, other)</code>
<code>></code>	<code>__gt__(self, other)</code>
<code>>=</code>	<code>__ge__(self, other)</code>

Важно: если `__ne__` не определён, Python автоматически инвертирует результат `__eq__` (но лучше определять явно). Для операторов порядка, если не определены, при сравнении будет возбуждено исключение `TypeError`.

Пример:

```
class Student:
```

```
    def __init__(self, name, grade):
```

```
        self.name = name
```

```
        self.grade = grade
```

```
    def __eq__(self, other):
```

```

        if not isinstance(other, Student):
            return NotImplemented
        return self.grade == other.grade

    def __lt__(self, other):
        if not isinstance(other, Student):
            return NotImplemented
        return self.grade < other.grade

    def __repr__(self):
        return f"Student({self.name}, {self.grade})"

alice = Student("Алиса", 4.5)
bob = Student("Боб", 4.0)
print(alice == bob)    # False
print(alice > bob)     # True (__gt__ генерируется из __lt__)

```

Примечание: Python умеет выводить `__gt__` из `__lt__`, `__le__` из `__gt__` и т.д. с помощью декоратора `functools.total_ordering`, но лучше реализовать все основные методы сравнения для полного контроля.

5. Методы контейнеров (эмуляция последовательностей и отображений)

Эти методы позволяют объекту вести себя как список, словарь или множество.

- `__len__(self)` — вызывается функцией `len()`. Должен возвращать целое число ≥ 0 .
- `__getitem__(self, key)` — доступ по индексу или ключу: `obj[key]`.
- `__setitem__(self, key, value)` — присваивание по индексу/ключу: `obj[key] = value`.
- `__delitem__(self, key)` — удаление элемента: `del obj[key]`.
- `__contains__(self, item)` — вызывается оператором `in`.
- `__iter__(self)` — должен возвращать итератор для объекта (используется в циклах `for`).

Пример простого «вектора» с доступом по индексу:

```

class MyVector:
    def __init__(self, *args):
        self._data = list(args)

```



```
def __len__(self):
```

```
    return len(self._data)
```

```
def __getitem__(self, idx):
```

```
    return self._data[idx]
```

```
def __setitem__(self, idx, value):
```

```
    self._data[idx] = value
```

```
def __contains__(self, item):
```

```
    return item in self._data
```

```
def __repr__(self):
```

```
    return f"MyVector({' ', ' '.join(map(str, self._data))})"
```

```
v = MyVector(1, 2, 3, 4)
```

```
print(len(v))          # 4
```

```
print(v[2])            # 3
```

```
v[1] = 10
```

```
print(2 in v)          # False
```

```
print(10 in v)         # True
```

6. Вызов объекта как функции (`__call__`)

Метод `__call__(self, ...)` позволяет экземпляру класса вести себя как функция. Это полезно для создания объектов с внутренним состоянием, которые можно «вызвать», например, для реализации декораторов или функторов.

```
class Multiplier:
```

```
    def __init__(self, factor):
```

```
        self.factor = factor
```

```
    def __call__(self, x):
```

```
        return x * self.factor
```

```
double = Multiplier(2)

print(double(5))      # 10

print(double(3.5))    # 7.0
```

7. Менеджеры контекста (`with`)

Магические методы `__enter__` и `__exit__` позволяют объекту использоваться в конструкции `with`. Это удобно для управления ресурсами (файлы, соединения, блокировки).

```
class ManagedFile:

    def __init__(self, filename, mode):

        self.filename = filename

        self.mode = mode

    def __enter__(self):

        self.file = open(self.filename, self.mode)

        return self.file

    def __exit__(self, exc_type, exc_val, exc_tb):

        self.file.close()

        # Если вернуть True, исключения будут подавлены (не рекомендуется
        # без нужды)

with ManagedFile('test.txt', 'w') as f:

    f.write('Hello, world!')

# файл автоматически закрыт после выхода из блока
```

Технические ограничения и особенности

1. **Не все операторы имеют магические методы.** Например, нельзя перегрузить логические операторы `and`, `or`, `not` напрямую — они используют приведение к булевой типу через `__bool__` (или `__len__` в старых версиях).
2. **Нельзя изменить приоритет операторов.** Порядок вычислений остаётся стандартным для Python.
3. **Нельзя изменить ассоциативность** (лево- или правоассоциативность).
4. **Магические методы должны быть определены в классе объекта, а не в его метаклассе** (за исключением некоторых методов типа `__new__`).

5. Рекомендуется возвращать `NotImplemented` при невозможности обработать операцию, а не возбуждать исключение сразу — это даёт шанс правому операнду обработать вызов.
6. Осторожность с бесконечной рекурсией: внутри метода `__getattr__` (перехват любого доступа к атрибутам) легко случайно вызвать рекурсию, если обращаться к `self.attr`.
7. Некоторые магические методы ищутся только в классе, а не в экземпляре. Например, `__hash__`, `__getattr__`, `__setattr__`. Это особенность реализации Python.

Сводная таблица основных магических методов

Метод	Назначение	Пример вызова	Принимает	Возвращает
<code>__init__(self, ...)</code>	Инициализация объекта	<code>obj = Class(...)</code>	Аргументы конструктора	<code>None</code>
<code>__new__(cls, ...)</code>	Создание объекта	Вызывается перед <code>__init__</code>	Класс и аргументы	Экземпляр (обычно)
<code>__del__(self)</code>	Деструктор	При сборке мусора	-	-
<code>__str__(self)</code>	Человеко-читаемая строка	<code>str(obj)</code> , <code>print(obj)</code>	-	<code>str</code>
<code>__repr__(self)</code>	Строка для разработчика	<code>repr(obj)</code> , в консоли	-	<code>str</code>
<code>__add__(self, other)</code>	Сложение <code>+</code>	<code>obj + other</code>	<code>other</code> — правый операнд	Любой объект
<code>__sub__(self, other)</code>	Вычитание <code>-</code>	<code>obj - other</code>	<code>other</code>	Любой объект
<code>__mul__(self, other)</code>	Умножение <code>*</code>	<code>obj * other</code>	<code>other</code>	Любой объект
<code>__truediv__(self, other)</code>	Деление <code>/</code>	<code>obj / other</code>	<code>other</code>	Любой объект

Метод	Назначение	Пример вызова	Принимает	Возвращает
<code>__floordiv__(self, other)</code>	Целочисленное деление <code>//</code>	<code>obj //</code> <code>other</code>	<code>other</code>	Любой объект
<code>__mod__(self, other)</code>	Остаток <code>%</code>	<code>obj %</code> <code>other</code>	<code>other</code>	Любой объект
<code>__pow__(self, other)</code>	Возведение в степень <code>**</code>	<code>obj **</code> <code>other</code>	<code>other</code>	Любой объект
<code>__radd__(self, other)</code>	Обратное сложение	<code>other +</code> <code>obj</code> (если <code>other</code> не поддерживает)	<code>other</code> — левый операнд	Любой объект
<code>__iadd__(self, other)</code>	Сложение с присваиванием <code>+=</code>	<code>obj +=</code> <code>other</code>	<code>other</code>	Обычно <code>self</code>
<code>__neg__(self)</code>	Унарный минус <code>-obj</code>	<code>-obj</code>	-	Любой объект
<code>__abs__(self)</code>	Абсолютное значение <code>abs(obj)</code>	<code>abs(obj)</code>	-	Число (обычно)
<code>__eq__(self, other)</code>	Равенство <code>==</code>	<code>obj ==</code> <code>other</code>	<code>other</code>	<code>bool</code> или <code>NotImplemented</code>
<code>__ne__(self, other)</code>	Неравенство <code>!=</code>	<code>obj !=</code> <code>other</code>	<code>other</code>	<code>bool</code> или <code>NotImplemented</code>
<code>__lt__(self, other)</code>	Меньше <code><</code>	<code>obj <</code> <code>other</code>	<code>other</code>	<code>bool</code> или <code>NotImplemented</code>

Метод	Назначение	Пример вызова	Принимает	Возвращает
<code>__le__(self, other)</code>	Меньше или равно <code><=</code>	<code>obj <= other</code>	<code>other</code>	<code>bool</code> или <code>NotImplemented</code>
<code>__gt__(self, other)</code>	Больше <code>></code>	<code>obj > other</code>	<code>other</code>	<code>bool</code> или <code>NotImplemented</code>
<code>__ge__(self, other)</code>	Больше или равно <code>>=</code>	<code>obj >= other</code>	<code>other</code>	<code>bool</code> или <code>NotImplemented</code>
<code>__len__(self)</code>	Длина <code>len(obj)</code>	<code>len(obj)</code>	-	<code>int</code> <code>>= 0</code>
<code>__getitem__(self, key)</code>	Доступ к элементу <code>obj[key]</code>	<code>obj[key]</code>	<code>key</code>	Любой объект
<code>__setitem__(self, key, value)</code>	Присваивание элементу	<code>obj[key] = value</code>	<code>key</code> , <code>value</code>	-
<code>__delitem__(self, key)</code>	Удаление элемента	<code>del obj[key]</code>	<code>key</code>	-
<code>__contains__(self, item)</code>	Проверка вхождения <code>item in obj</code>	<code>item in obj</code>	<code>item</code>	<code>bool</code>
<code>__iter__(self)</code>	Получение итератора	<code>iter(obj)</code> , <code>for x in obj:</code>	-	Итератор
<code>__next__(self)</code>	Следующий элемент итератора	<code>next(obj)</code>	-	Следующий элемент или <code>StopIteration</code>

Метод	Назначение	Пример вызова	Принимает	Возвращает
<code>__call__(self, . ..)</code>	Вызов объекта как функции	<code>obj(...)</code>	Любые аргументы	Любой объект
<code>__enter__(self)</code>	Вход в контекстный менеджер	<code>with obj:</code>	-	Любой объект (обычно <code>self</code>)
<code>__exit__(self, exc_type, exc_val, exc_tb)</code>	Выход из контекстного менеджера	При завершении <code>with</code>	Тип исключения, значение, <code>traceback</code>	<code>bool</code> (если <code>True</code> , подавить исключение)
<code>__hash__(self)</code>	Получение хеша для словарей/множеств	<code>hash(obj)</code> , при добавлении в <code>set/dict</code>	-	<code>int</code>
<code>__bool__(self)</code>	Приведение к булеву значению	<code>bool(obj)</code> , в условиях	-	<code>bool</code>
<code>__getattr__(self, name)</code>	Доступ к несуществующему атрибуту	<code>obj.attr</code> (если <code>attr</code> нет)	<code>name</code> (строка)	Любой объект
<code>__setattr__(self, name, value)</code>	Присваивание атрибута	<code>obj.attr = value</code>	<code>name</code> , <code>value</code>	-
<code>__delattr__(self, name)</code>	Удаление атрибута	<code>del obj.attr</code>	<code>name</code>	-

Где найти полный список?

Официальная документация Python содержит исчерпывающий список всех магических методов в разделе **Data Model** (Модель данных). Рекомендуется изучить её для глубокого понимания возможностей языка.

🔗 [Специальные методы Python — Data model \(docs.python.org\)](https://docs.python.org/3/reference/datamodel.html)

В этом документе описаны все методы, их сигнатуры и семантика. Это ваш основной справочник при работе с магическими методами.